

Personal Trading Intelligence Platform

Complete System Blueprint v2.0

Author	Jaiminkumar Patel
Date	June 2026
Status	Design Complete — Ready for Implementation
Architecture	MCP-first (Claude Desktop) → Dashboard UI (Phase 4+)
Hardware	MacBook Pro M5 Pro 48GB + VPS for always-on
Primary Broker	Webull (OpenAPI) — Robinhood later via SnapTrade
Components	14 services + 14 database tables
Baseline Rules	50 pre-loaded options strategy rules
Est. Timeline	24 weeks (6 months at 10-15 hrs/week)

CONTEXT FOR AI CONTINUATION: This is the complete blueprint for a personal trading intelligence platform. All design decisions have been reviewed and approved. Implementation starts at Phase 1 Week 1. The system uses MCP (Model Context Protocol) as the primary interface — Claude Desktop is the client for Phases 1-3, with a custom Next.js dashboard added in Phase 4+. 14 components, 14 database tables, 50 baseline strategy rules, position monitoring with real-time alerts, and a self-learning engine that personalizes over time. Read this entire document before making any changes.

Table of Contents

1. Project Vision & Goals
2. Phase Roadmap
3. System Architecture
4. Component Map (14 Components)
5. Component Details
6. Database Schema (14 Tables)
7. Security Architecture
8. Baseline Strategy Rules (50 Rules)
9. Position Lifecycle & Monitoring
10. Notification System
11. Self-Learning Loop
12. Token Usage Strategy
13. Technology Stack
14. Data Sources & Subscriptions
15. Implementation Roadmap (24 Weeks)
16. Dashboard UI (Phase 4+)
17. Open Decisions

1. Project Vision & Goals

Build a personal trading intelligence platform that evolves from a portfolio tracker into an autonomous, self-improving options trading algorithm personalized to each user's risk tolerance, budget, and execution behavior.

Core Requirements

- **Portfolio Tracking** — live positions from Webull, real-time P&L, sell signals for max profit
- **Options Discovery** — weekly scan of 3000+ optionable stocks, filter market cap > \$50M, no penny stocks
- **Strategy Selection** — system autonomously decides best strategy (covered call, credit spread, iron condor, straddle, etc.)
- **Specific Contracts** — exact strike price, entry price, stop loss, sell target
- **Position Monitoring** — real-time tracking with profit target and stop loss alerts
- **Self-Validation** — tracks every prediction, measures win/loss ratio, learns from results
- **Multi-User** — invite system, each user gets their own profile, predictions, and learning
- **Autonomous Evolution** — eventually executes trades with user approval

Design Principles

- **MCP-first architecture** — Claude Desktop as primary client, custom UI added later
- **Minimize cloud LLM cost** — local LLM for analysis, cloud only for conversation
- **Security first** — OAuth2, encrypted credentials, read-only by default, audit log
- **Self-improving** — starts with 50 baseline rules, learns and adjusts from outcomes
- **Build own RAG** — SEC filings, earnings transcripts, news for fundamental context

User Profile

- **Primary broker:** Webull (heavy usage)
- **Secondary broker:** Robinhood (occasional, added later via SnapTrade)
- **Options focus:** All strategies — system recommends which fits best
- **Ticker universe:** System discovers autonomously (market cap > \$50M, price > \$5)

2. Phase Roadmap

Phase	Name	Weeks	Description	Key Deliverables
1	SEE	1-6	Portfolio tracking + sell signals	Webull integration, live positions, P&L, exit signals, MCP server
2	THINK	7-14	Discovery + strategy + recommendations	Scanner, strategy engine, RAG, position monitor, alerts
3	LEARN	15-20	Self-improving algorithm	Prediction tracking, weight adjustment, user profile learning, backtesting
4	ACT	21-24	Execution + Dashboard UI	Trade execution via Webull, Next.js dashboard, deployment

Each phase builds on the previous. Phase 1 infrastructure is reused in all subsequent phases. Claude Desktop is the client for Phases 1-3. Dashboard UI added in Phase 4.

3. System Architecture

Phase 1-3: Claude Desktop as Only Client

You → Claude Desktop → MCP Protocol → MCP Server → Backend Services

Phase 4+: Add Dashboard as Second Client

Claude Desktop ████
████ MCP Server ███ Backend Services
Dashboard ██████████

Same MCP server, same tools. Dashboard connects as an MCP client. No REST API layer needed. Only addition is a WebSocket connection for real-time alert push notifications.

Architecture Layers

Layer	Purpose	Components
Client	User interaction	Claude Desktop (Phase 1-3) Next.js Dashboard (Phase 4+)
MCP Server	Orchestration, tool routing	Python (FastMCP) or Go 18+ tool endpoints
Broker	Personal financial data	Webull OpenAPI (OAuth2) SnapTrade for Robinhood (later)
Market Data	Prices, options, technicals	Polygon.io (\$29/mo) Yahoo Finance (free backup)
Research (RAG)	Fundamental context	SEC EDGAR, earnings transcripts ChromaDB + local embeddings
Intelligence	Analysis and reasoning	Local LLM (Qwen 14B) Technical analysis engine Strategy engine
Monitoring	Position tracking, alerts	Position Monitor service Notification service (email/Slack/SMS)
Storage	Persistence	PostgreSQL (encrypted) ChromaDB for RAG embeddings

MCP Tool Endpoints

Tool	Phase	Description
get_positions	1	Live portfolio positions from Webull
get_balances	1	Account cash, buying power, margin
get_orders	1	Open and filled order history
get_sell_signals	1	When to exit current positions for max profit
get_portfolio_pnl	1	Overall portfolio performance and P&L
discover_options	2	Weekly scan — top 10-20 options opportunities
analyze_ticker	2	Deep dive on one ticker (technicals + fundamentals)
get_recommendation	2	Specific contract: strategy, strike, entry, SL, target
get_options_chain	2	Full options chain with greeks for a ticker

get_market_regime	2	Current market conditions (bull/bear/sideways, VIX)
explain_strategy	2	Natural language explanation of a recommendation
activate_position	2	User entered a trade — start monitoring
get_active_positions	2	All currently monitored positions
get_prediction_log	3	Historical predictions and their outcomes
get_accuracy_report	3	Win/loss ratio, Sharpe, per strategy performance
get_user_profile	3	Risk tolerance, budget, execution behavior
backtest_strategy	3	Backtest a strategy on historical data
get_learnings	3	What the system has learned about what works
execute_trade	4	Place a trade on Webull (with user approval)
adjust_algorithm	4	Manually adjust strategy weights

4. Component Map — 14 Components

#	Component	Purpose	Phase	Depends On
1	MCP Server	Entry point, tool routing	1	All components
2	Broker Connector	Webull API (positions, orders, P&L)	1	Webull OAuth2
3	Market Data Service	Quotes, options chains, historical	1	Polygon.io API
4	Options Flow Service	Unusual activity, institutional flow	2	Unusual Whales API
5	Scanner / Discovery	Weekly scan, filter, rank candidates	2	C3, C4, C6
6	Technical Analysis	Trend, S/R, RSI, MACD, patterns	1	C3 (price data)
7	Strategy Engine	Decide strategy + specific contracts	2	C6, C8, C9
8	RAG Pipeline	Ingest docs, embed, query context	2	ChromaDB, Ollama
9	Local LLM Service	Ollama + Qwen 14B for reasoning	1	Ollama
10	Prediction Tracker	Log every recommendation + outcome	2	C12 (Postgres)
11	Learning Engine	Pattern detection, weight adjustment	3	C10, C12
12	Postgres Database	All persistence, encrypted creds	1	None
13	Position Monitor	Track active positions, trigger alerts	2	C3, C12, C14
14	Notification Service	Deliver alerts (email/Slack/SMS)	2	C13

Component Interaction Flow

```
Claude calls MCP Server
MCP Server → C5: Scanner (weekly scan)
C5 → C3: Market Data (price, volume, IV for 3000+ stocks)
C5 → C4: Options Flow (unusual activity)
C5 → C6: Technical Analysis (trend, S/R, momentum)
C5 filters: market cap > $50M, price > $5, options liquidity
C5 returns top 10-20 candidates
MCP Server → C7: Strategy Engine (for each candidate)
C7 → C6: Technicals (detailed analysis)
C7 → C8: RAG (earnings, news, SEC filings)
C7 → C9: Local LLM (reasoning about best strategy)
C7 returns: strategy, strike, entry, stop loss, target
MCP Server → C10: Prediction Tracker (log recommendation)
MCP Server returns results to Claude
Claude presents recommendations to you
```

5. Component Details

C1 — MCP Server

- **Language:** Python (FastMCP) or Go
- **Protocol:** MCP (Model Context Protocol) over SSE
- **Deployment:** VPS for always-on, local MacBook for dev
- **Auth:** API key per user, hashed in DB
- **Tools:** 20 endpoints across all 4 phases

C2 — Broker Connector

- **Webull OpenAPI:** OAuth2, Python SDK, read-only (Phase 1-3), trade (Phase 4)
- **Endpoints:** positions, balances, orders, account info, trade execution
- **Security:** tokens encrypted at rest (pgcrypto), auto-refresh
- **SnapTrade:** for Robinhood later — aggregator handles token management
- **Scopes:** read-only by default, write requires user re-authorization

C3 — Market Data Service

- **Primary:** Polygon.io (\$29/mo) — quotes, options chains, greeks, historical, unusual activity
- **Backup:** Yahoo Finance (free) — basic quotes and options
- **Protocols:** HTTP for standard, MQTT for real-time streaming
- **Caching:** local cache to reduce API calls, TTL-based eviction
- **Data:** real-time quotes, OHLCV bars (1min-monthly), full options chains with all greeks

C4 — Options Flow Service

- **Source:** Unusual Whales or FlowAlgo (\$40-70/mo)
- **Signals:** volume spikes, large block trades, dark pool activity, P/C ratio shifts
- **Schedule:** poll on schedule, cache and score results
- **Integration:** feeds into Scanner scoring

C5 — Scanner / Discovery Engine

- **Universe:** 3000+ optionable US stocks
- **Filters:** market cap > \$50M, price > \$5, min options OI and volume
- **Scoring:** combines technicals + IV rank + unusual flow + earnings proximity
- **Output:** ranked top 10-20 candidates with reasons
- **Schedule:** weekly full scan (Sunday), daily quick scan (pre-market)
- **Runs locally:** pure computation + local LLM, no cloud API

C6 — Technical Analysis Engine

- **Indicators:** MA (20/50/200), EMA, MACD, RSI, Bollinger, ATR, VWAP
- **Patterns:** S/R levels, trend channels, breakout/breakdown detection
- **Volume:** volume profile, relative volume, accumulation/distribution
- **Library:** ta-lib or pandas-ta
- **Performance:** milliseconds per ticker, no LLM needed

C7 — Strategy Engine

- **Input:** ticker, technicals, fundamentals (RAG), market regime, IV data, user profile
- **Logic:** match conditions → score matching rules → pick highest weighted strategy
- **Output:** strategy, specific strikes, expiry, entry, SL, target, risk/reward, position size

- **Local LLM:** assists with edge case reasoning and natural language explanation
- **User profile aware:** respects max loss, max positions, risk tolerance, excluded strategies

C8 — RAG Pipeline

- **Sources:** SEC EDGAR (10-K, 10-Q, 8-K), earnings transcripts, financial news
- **Vector DB:** ChromaDB (local, free)
- **Embeddings:** nomic-embed-text via Ollama (local, free)
- **Chunking:** document-aware splitting (sections, paragraphs, tables)
- **Update:** SEC filings on publish, earnings quarterly, news daily

C9 — Local LLM Service

- **Runtime:** Ollama on MacBook M5 Pro 48GB
- **Model:** Qwen 14B for analysis reasoning, Qwen 2.5 Coder 14B for dev
- **Use cases:** strategy reasoning, ticker summaries, natural language explanations
- **NOT used for:** scanning (pure compute), technicals (math), options pricing (formulas)

C10 — Prediction Tracker

- **Logs on every recommendation:** ticker, strategy, strikes, entry, SL, target, IV rank, market regime, VIX, sector, earnings proximity, flow signals, confidence, reasoning
- **Tracks outcomes:** actual P&L, hit target or SL, days held, actual vs predicted move
- **Metrics:** win rate, avg return, max drawdown, Sharpe, profit factor
- **Granularity:** per strategy, per market regime, per IV env, per sector, per user

C11 — Learning Engine

- **Pattern detection:** finds what works under what conditions
- **Weight adjustment:** win → increase rule weight, loss → decrease
- **User profile learning:** tracks execution behavior, adjusts recommendations
- **Market regime awareness:** different strategies for different environments
- **Minimum data:** 20+ trades before adjustments, 200+ for full personalization

C12 — Postgres Database

- **Tables:** 14 tables across users, broker, market, predictions, learning, notifications
- **Encryption:** pgcrypto for all credentials and tokens
- **Encryption key:** environment variable only, never in DB or code

C13 — Position Monitor

- **Schedule:** every 1-2 min during market hours, every 5 min pre/post market
- **Fetches:** current option prices from Polygon.io / Webull
- **Updates:** active_positions with current mark, P&L, greeks
- **Snapshots:** position_snapshots every 15 min for P&L, charts
- **Auto-detect:** matches new Webull positions to pending predictions
- **On close:** creates outcome record, triggers learning engine

C14 — Notification Service

- **Channels:** email, SMS, Slack, in-app (WebSocket for dashboard)
- **Alert types:** profit target hit/approaching, stop loss hit/approaching, DTE warning, earnings warning, unusual flow against position

- **User prefs:** per-user channel config, quiet hours, threshold customization
- **Severity:** info, warning, critical

6. Database Schema — 14 Tables

All tables use UUID primary keys. Multi-user from day one (user_id on every table). Broker tokens encrypted with pgcrypto. API keys stored as SHA-256 hashes.

Group 1: Users & Auth

```
-- TABLE: users
id UUID PK, email VARCHAR UNIQUE, display_name VARCHAR,
is_active BOOLEAN, invited_by UUID FK(users), created_at, updated_at

-- TABLE: user_profiles
id UUID PK, user_id UUID FK UNIQUE,
starting_capital DECIMAL, max_loss_per_trade DECIMAL,
max_loss_per_trade_pct DECIMAL, max_open_positions INT,
risk_tolerance VARCHAR (conservative|moderate|aggressive),
preferred_strategies JSONB, excluded_strategies JSONB,
preferred_dte_min INT, preferred_dte_max INT,
-- Learned over time (auto-updated):
avg_hold_days DECIMAL, early_exit_rate DECIMAL,
best_performing_strategy VARCHAR, worst_performing_strategy VARCHAR

-- TABLE: user_api_keys
id UUID PK, user_id UUID FK, api_key_hash VARCHAR(64),
label VARCHAR, scopes JSONB, is_active BOOLEAN,
last_used_at TIMESTAMP, expires_at TIMESTAMP

-- TABLE: invites
id UUID PK, invited_by UUID FK, email VARCHAR,
invite_code VARCHAR UNIQUE, status VARCHAR (pending|accepted|expired),
expires_at TIMESTAMP, accepted_at TIMESTAMP
```

Group 2: Broker Connections

```
-- TABLE: broker_connections
id UUID PK, user_id UUID FK, broker_name VARCHAR (webull|robinhood),
auth_method VARCHAR (oauth2|snaptrade),
access_token BYTEA (encrypted), refresh_token BYTEA (encrypted),
token_expiry TIMESTAMP,
snaptrade_user_id VARCHAR, snaptrade_user_secret BYTEA,
is_active BOOLEAN, last_synced_at TIMESTAMP
UNIQUE(user_id, broker_name)
```

Group 3: Market Context

```
-- TABLE: market_regimes
id UUID PK, captured_at TIMESTAMP UNIQUE,
spy_trend VARCHAR, spy_rsi DECIMAL, spy_above_200ma BOOLEAN,
vix_level DECIMAL, vix_trend VARCHAR, vix_percentile DECIMAL,
pct_above_50ma DECIMAL, pct_above_200ma DECIMAL,
advance_decline INT, leading_sector VARCHAR, lagging_sector VARCHAR,
ten_yr_yield DECIMAL, fed_funds_rate DECIMAL

-- TABLE: ticker_snapshots
id UUID PK, ticker VARCHAR, captured_at TIMESTAMP,
price DECIMAL, market_cap BIGINT, sector VARCHAR, industry VARCHAR,
trend VARCHAR, rsi_14 DECIMAL, macd_signal VARCHAR,
above_20ma BOOLEAN, above_50ma BOOLEAN, above_200ma BOOLEAN,
atr_14 DECIMAL, relative_volume DECIMAL,
support_level DECIMAL, resistance_level DECIMAL,
iv_rank DECIMAL, iv_percentile DECIMAL,
put_call_ratio DECIMAL, options_volume BIGINT,
```

```
next_earnings_date DATE, days_to_earnings INT,
has_unusual_flow BOOLEAN, flow_direction VARCHAR, flow_details JSONB
UNIQUE(ticker, captured_at)
```

Group 4: Predictions & Outcomes (Core)

```
-- TABLE: predictions
id UUID PK, user_id UUID FK, ticker_snapshot_id UUID FK,
market_regime_id UUID FK,
ticker VARCHAR, strategy VARCHAR, direction VARCHAR,
legs JSONB [{type, strike, expiry, premium, delta, contracts}],
entry_credit_debit DECIMAL, max_profit DECIMAL, max_loss DECIMAL,
risk_reward_ratio DECIMAL, breakeven_prices JSONB,
profit_target_pct DECIMAL, stop_loss_pct DECIMAL,
profit_target_price DECIMAL, stop_loss_price DECIMAL,
expiry_date DATE, days_to_expiry INT,
iv_rank_at_entry DECIMAL, price_at_entry DECIMAL, vix_at_entry DECIMAL,
confidence_score DECIMAL, reasoning TEXT, baseline_rule_used VARCHAR,
status VARCHAR (pending|active|closed_profit|closed_loss|expired|cancelled),
user_action VARCHAR (followed|modified|skipped|exited_early),
recommended_at TIMESTAMP, activated_at TIMESTAMP, closed_at TIMESTAMP

-- TABLE: outcomes
id UUID PK, prediction_id UUID FK UNIQUE, user_id UUID FK,
actual_pnl DECIMAL, actual_pnl_pct DECIMAL,
hit_target BOOLEAN, hit_stop_loss BOOLEAN, expired_worthless BOOLEAN,
price_at_close DECIMAL, price_change_pct DECIMAL,
iv_at_close DECIMAL, iv_change DECIMAL,
days_held INT, closed_reason VARCHAR,
followed_plan BOOLEAN, exit_reason_notes TEXT,
vix_at_close DECIMAL, spy_change_pct DECIMAL
```

Group 5: Active Position Monitoring

```
-- TABLE: active_positions
id UUID PK, user_id UUID FK, prediction_id UUID FK,
ticker VARCHAR, strategy VARCHAR, legs JSONB,
entry_type VARCHAR (manual|auto_detected|executed),
actual_entry_credit_debit DECIMAL, actual_entry_date TIMESTAMP,
profit_target_price DECIMAL, stop_loss_price DECIMAL,
profit_target_pct DECIMAL, stop_loss_pct DECIMAL,
trail_stop_enabled BOOLEAN, trail_stop_pct DECIMAL,
expiry_date DATE, alert_at_dte INT, close_before_earnings BOOLEAN,
current_mark_price DECIMAL, current_pnl DECIMAL, current_pnl_pct DECIMAL,
max_profit_seen DECIMAL, max_loss_seen DECIMAL, last_monitored_at TIMESTAMP,
status VARCHAR (active|closing|closed), closed_at TIMESTAMP,
close_reason VARCHAR

-- TABLE: position_snapshots (for P&L; charts)
id UUID PK, position_id UUID FK,
mark_price DECIMAL, pnl DECIMAL, pnl_pct DECIMAL,
underlying_price DECIMAL, iv_current DECIMAL,
delta_current DECIMAL, theta_current DECIMAL,
dte_remaining INT, captured_at TIMESTAMP
```

Group 6: Learning Engine

```
-- TABLE: strategy_weights
id UUID PK, user_id UUID FK,
strategy VARCHAR, condition_key VARCHAR,
```

```

condition_desc TEXT,
base_weight DECIMAL, learned_weight DECIMAL, final_weight DECIMAL,
total_trades INT, wins INT, losses INT,
avg_return_pct DECIMAL, win_rate DECIMAL
UNIQUE(user_id, strategy, condition_key)
-- TABLE: learning_log
id UUID PK, user_id UUID FK, strategy_weight_id UUID FK,
previous_weight DECIMAL, new_weight DECIMAL,
trigger_prediction_id UUID FK, reason TEXT

```

Group 7: Notifications & Audit

```

-- TABLE: notifications
id UUID PK, user_id UUID FK, position_id UUID FK,
alert_type VARCHAR, severity VARCHAR (info|warning|critical),
title VARCHAR, message TEXT, data JSONB,
channels JSONB, delivered_via JSONB, read_at TIMESTAMP,
user_action VARCHAR (acknowledged|acted_on|dismissed|snoozed)
-- TABLE: notification_preferences
id UUID PK, user_id UUID FK UNIQUE,
email VARCHAR, phone VARCHAR, slack_webhook TEXT,
notify_profit_target BOOLEAN, notify_stop_loss BOOLEAN,
notify_dte_warning BOOLEAN, notify_earnings_warning BOOLEAN,
notify_unusual_flow BOOLEAN, notify_weekly_scan BOOLEAN,
profit_approaching_pct DECIMAL, stop_approaching_pct DECIMAL,
quiet_start TIME, quiet_end TIME, timezone VARCHAR
-- TABLE: audit_log
id UUID PK, user_id UUID FK,
action VARCHAR, resource VARCHAR,
ip_address INET, user_agent TEXT, details JSONB
-- TABLE: rag_documents
id UUID PK, source_type VARCHAR, ticker VARCHAR,
title TEXT, source_url TEXT UNIQUE, published_at TIMESTAMP,
ingested_at TIMESTAMP, chunk_count INT, chromadb_collection VARCHAR

```

7. Security Architecture

Layer	What	How
OAuth2 Flow	User connects broker	Redirect to Webull login. Your server NEVER sees their password.
Token Encryption	Access/refresh tokens	pgcrypto symmetric encryption. Key in env var, never in DB/code.
API Key Auth	MCP server access	SHA-256 hash stored. Plaintext never persisted. Scoped: read/trade.
Read-Only Default	Phase 1-3	OAuth2 read scopes only. No trade/transfer capability.
Data Isolation	Multi-user	Every query filtered by user_id. No cross-user data access.
Audit Log	All access	Every broker read, trade, settings change logged.
User Revocation	Broker disconnect	User can revoke from Webull side. App detects and marks inactive.
Quiet Hours	Notifications	No alerts during user-defined sleep hours.

ZERO secrets on the server: No Webull passwords. No plaintext tokens in DB. No API keys in code. No encryption keys in DB. Encryption key in environment variable only. Even if database is breached, attacker gets encrypted blobs and read-only scoped tokens that cannot move money.

8. Baseline Strategy Rules — 50 Rules

Pre-loaded into strategy_weights table for every new user. System starts smart from day one. Weights adjust as the learning engine tracks outcomes. base_weight is fixed, learned_weight changes, final_weight = base_weight + learned_weight.

Category 1: IV-Based (10 rules)

#	Condition	Strategy	Wt
1	IV rank > 60, sideways	Iron Condor	85
2	IV rank > 60, sideways	Credit Spread (put)	80
3	IV rank > 60, mild up	Credit Spread (put)	78
4	IV rank > 60, mild down	Credit Spread (call)	75
5	IV rank > 80	Iron Condor	90
6	IV rank > 80	Short Strangle	82
7	IV rank < 30, strong up	Long Call	75
8	IV rank < 30, strong down	Long Put	72
9	IV rank < 30, strong up	Debit Spread (call)	78
10	IV rank < 30, strong down	Debit Spread (put)	76

Category 2: Trend-Based (8 rules)

#	Condition	Strategy	Wt
11	Strong uptrend, above all MAs	Credit Spread (put)	80
12	Strong downtrend, below all MAs	Credit Spread (call)	78
13	Breakout above resistance + volume	Long Call	75
14	Breakdown below support + volume	Long Put	73
15	Price at major support, bouncing	Debit Spread (call)	72
16	Price at resistance, rejecting	Debit Spread (put)	70
17	Sideways 20+ days, tight range	Iron Condor	82
18	Sideways, Bollinger compression	Straddle	65

Category 3: Earnings (6 rules)

#	Condition	Strategy	Wt
19	Earnings < 7d, IV > 70	AVOID	90
20	Earnings < 7d, IV > 70	Iron Condor (post-earnings)	70
21	Earnings < 3d, big move expected	Straddle	60
22	Earnings < 3d, big move expected	Strangle	58
23	1d post-earnings, IV crushed	Credit Spread	75
24	Earnings < 7d, any strategy	Reduce size 50%	85

Category 4: VIX / Market Regime (8 rules)

#	Condition	Strategy	Wt
---	-----------	----------	----

25	VIX > 30 (fear)	Cash Secured Put	80
26	VIX > 30	Wider strikes all strategies	85
27	VIX > 30	Reduce position size 50%	88
28	VIX < 15 (complacency)	Iron Condor (narrow)	70
29	VIX < 15	Long Straddle	55
30	VIX 15-25 (normal)	All strategies eligible	70
31	VIX rising sharply (+20%/wk)	Long Put / Debit Spread put	72
32	VIX falling from spike	Credit Spread (put)	78

Category 5: Greeks (6 rules)

#	Condition	Strategy	Wt
33	DTE < 21, theta accelerating	Sell premium strategies	80
34	DTE > 45, theta slow	Buy premium strategies	70
35	Delta < 0.30 on short strikes	Credit Spread / Iron Condor	82
36	Delta < 0.16 on short strikes	Iron Condor	85
37	High vega + IV > 60	Sell premium (short vega)	80
38	High vega + IV < 30	Buy premium (long vega)	72

Category 6: Unusual Flow (5 rules)

#	Condition	Strategy	Wt
39	Bullish call volume > 3x avg	Long Call / Debit Spread call	70
40	Bearish put volume > 3x avg	Long Put / Debit Spread put	68
41	Large block trade calls, near money	Long Call	65
42	P/C ratio > 1.5 (extreme bearish)	Contrarian: Credit Spread put	60
43	P/C ratio < 0.5 (extreme bullish)	Contrarian: Credit Spread call	58

Category 7: Risk Management (7 rules — Always Active)

#	Condition	Action	Wt
44	Any trade	Max loss = user's max_loss_per_trade	95
45	Positions >= max_open	Skip new trades	95
46	Same sector > 40%	Diversify	85
47	Any credit trade	Take profit at 50% max profit	80
48	Any trade	Close at 21 DTE if not at target	75
49	Loss > 2x credit received	Close immediately	90
50	3 consecutive losses same strategy	Reduce weight, pause strategy	80

9. Position Lifecycle & Monitoring

Flow: Recommendation → Entry → Monitor → Close → Learn

- 1. System recommends trade (logged in predictions table)
- 2. User enters trade:
 - Way A: Tells Claude 'I bought the NVDA iron condor at \$2.45'
 - activate_position MCP tool → creates active_position
 - Way B: Auto-detected via Webull API position matching
 - new position matches pending prediction → auto-activate
- 3. Position Monitor starts watching (every 1-2 min market hours)
- 4. Alerts trigger on conditions:
 - Current P&L; >= profit target → NOTIFY: Take profit
 - Current P&L; <= -stop loss → NOTIFY: Exit now
 - Profit retreating from peak → trailing stop check
 - DTE <= alert threshold → NOTIFY: Gamma risk, consider closing
 - Earnings approaching → NOTIFY: Close or hold decision
 - Unusual flow against position → NOTIFY: Adverse signal
- 5. User closes position (or expires/assigned)
- 6. Outcome recorded in outcomes table
- 7. Learning engine updates strategy_weights based on result

Position Monitor Schedule

Time Period	Frequency	What
Market hours (9:30-16:00 ET)	Every 1-2 minutes	Full price check, P&L update, alert check
Pre/post market	Every 5 minutes	Price check, major alert check only
Weekends/holidays	Off	No monitoring
Every 15 minutes (market hours)	Snapshot	Save position_snapshot for P&L charts

Alert Notification Example

Mon 10:00 – Enter NVDA iron condor at \$2.40 credit
Tue 14:30 – Position at +35% → No alert (below threshold)
Wed 11:00 – Position at +42% → INFO: Approaching target (target: 50%)
Wed 15:00 – Position at +51% → CRITICAL: HIT TARGET. Close position.
--- OR ---
Wed 11:00 – Position at -150% → WARNING: Approaching stop loss (stop: -200%)
Wed 14:00 – Position at -200% → CRITICAL: HIT STOP. Exit immediately.

10. Notification System

Alert Type	Severity	Trigger	Action
profit_target_approaching	info	P&L >= 80% of target	Consider closing
profit_target_hit	critical	P&L >= target	Close position
stop_loss_approaching	warning	Loss >= 80% of stop	Prepare to exit
stop_loss_hit	critical	Loss >= stop loss	Exit immediately
trailing_stop_hit	critical	Profit dropped from peak by trail_stop_pct	Close to lock gains
dte_warning	warning	DTE <= alert_at_dte	Close or roll
earnings_warning	warning	Earnings within 3 days	Close or hold decision
unusual_flow_against	info	Large adverse flow detected	Review position
position_update	info	Auto-detected entry from Webull	Confirm monitoring
weekly_scan_ready	info	Weekly scan complete	Review recommendations

Delivery Channels

Channel	Use Case	Implementation
Email	All alerts, weekly summaries	SMTP or SendGrid
SMS	Critical alerts only (stop loss, target)	Twilio
Slack	All alerts, configurable channel	Webhook
In-app (WebSocket)	Dashboard real-time (Phase 4)	WebSocket hub
Claude Desktop	Conversational alerts	MCP notification tool

11. Self-Learning Loop

Every recommendation is tracked. Every outcome is measured. The algorithm adjusts based on what actually works. This is what separates this system from a simple screener.

Learning Timeline

Timeframe	Data Points	What Happens
Week 1-4	10-20	Collecting baseline data. No adjustments yet.
Month 2-3	50+	Patterns emerge. Basic strategy weighting begins. 'Iron condors on high IV win 72%'
Month 3-6	100-200	Weights refined. Market regime awareness. User execution patterns detected.
Month 6+	200+	Fully personalized. Knows YOUR risk tolerance, execution style. Tailored recommendations.

Learning Dimensions

- **By strategy:** iron condor win rate vs credit spread vs straddle vs long calls
- **By IV environment:** high IV (>60) vs medium (30-60) vs low (<30)
- **By market regime:** trending up vs down vs sideways
- **By VIX level:** low (<15) vs normal (15-25) vs high (>25)
- **By sector:** tech, healthcare, energy, financials — each behaves differently
- **By earnings proximity:** pre-earnings, post-earnings, no earnings nearby
- **By DTE:** weeklies vs monthlies vs LEAPs
- **By user execution:** followed plan, exited early, modified, skipped

Weight Adjustment Logic

```
On WIN:
adjustment = base_adjustment * confidence_multiplier
learned_weight += adjustment (typically +1 to +3 points)
High confidence + win = bigger positive adjustment

On LOSS:
adjustment = base_adjustment * confidence_multiplier
learned_weight -= adjustment (typically -1 to -3 points)
High confidence + loss = bigger negative adjustment

final_weight = base_weight + learned_weight
Example: base=85, learned=+15 after 30 wins → final=100 (strong signal)
Example: base=85, learned=-25 after 20 losses → final=60 (weak, deprioritized)
```

12. Token Usage Strategy

90% of computation runs locally. Cloud LLM is only the conversation interface.

Task	Where It Runs	Cost
Your conversation with Claude	Cloud (Claude API)	~\$5-10/month
Ticker scanning (3000+ stocks)	Local (pure computation)	Free
Technical analysis	Local (Python, ta-lib)	Free
Options chain math / Greeks	Local (Python)	Free
Strategy reasoning	Local LLM (Qwen 14B)	Free
RAG embedding	Local (nomic-embed-text)	Free
RAG query + context retrieval	Local (ChromaDB)	Free
Natural language explanations	Local LLM (Qwen 14B)	Free
Prediction tracking	Local (Postgres)	Free
Learning engine	Local (Python + stats)	Free
Position monitoring	Local (Postgres + Polygon API)	Free (Polygon sub)

13. Technology Stack

Category	Technology	Why
Language	Python (primary)	Best ecosystem for finance + ML + APIs
MCP Server	Python (FastMCP) or Go	FastMCP for speed of dev, Go for performance
Database	PostgreSQL + pgcrypto	Reliable, encrypted, JSON support
Vector DB	ChromaDB	Local, free, Python-native
Local LLM	Ollama + Qwen 14B	Fits in 48GB, strong reasoning
Coding LLM	Qwen 2.5 Coder 14B	Local coding assistant for development
Embeddings	nomic-embed-text (Ollama)	Local, free, good for finance docs
Technical Analysis	ta-lib or pandas-ta	Industry standard indicators
Options Pricing	py_vollib or mibian	Black-Scholes, greeks calculation
Data Processing	pandas, numpy	Standard data manipulation
Market Data	Polygon.io Python SDK	Official SDK, well documented
Broker	Webull OpenAPI SDK	Official Python SDK
HTTP Framework	FastAPI	For REST endpoints if needed
Notifications	SendGrid (email) + Twilio (SMS)	Reliable, free tier available
Containerization	Docker + Docker Compose	Postgres, ChromaDB, Ollama, app
Deployment	VPS (DigitalOcean/Hetzner)	Always-on for monitoring + alerts
TLS	Traefik	Reverse proxy, auto SSL certs
Dashboard (Phase 4)	Next.js + React + Tailwind	MCP client + WebSocket for alerts
Charts (Phase 4)	Recharts	P&L charts, performance graphs
Dev Machine	MacBook M5 Pro 48GB	Local LLM + full dev stack

14. Data Sources & Subscriptions

Service	Cost/mo	What You Get	Phase	Required?
Webull OpenAPI	Free	Positions, orders, balances, trade exec	1	Yes
Polygon.io	\$29	Real-time quotes, options chains, greeks, historical, unusual activity	1	Yes
Unusual Whales or FlowAlgo	\$40-70	Options flow, institutional activity, dark pool data	2	Recommended
Seeking Alpha or TipRanks	\$20-30	Analyst ratings, earnings estimates, research reports for RAG	2	Optional
SEC EDGAR	Free	10-K, 10-Q, 8-K filings	2	Yes
Yahoo Finance	Free	Backup quotes, basic options chains	1	Free backup
SnapTrade	Free tier	Robinhood integration	Future	Optional
SendGrid	Free tier	Email notifications	2	Yes
Twilio	~\$1-5	SMS for critical alerts	2	Optional
VPS Hosting	\$10-20	Always-on server	1	Yes

Total estimated monthly cost: ~\$110-155/month for institutional-grade data + hosting. Compare that to one bad options trade.

15. Implementation Roadmap — 24 Weeks

Phase 1: SEE (Weeks 1-6)

Week	Focus	Build	Test
1	Foundation	Postgres setup, all CREATE TABLE scripts, User auth (JWT/API key), invite system, Docker Compose (Postgres+ChromaDB+Ollama)	Create user, generate API key, invite another user
2	Broker	Webull OAuth2 flow, token encryption, token refresh, get_positions, get_balances, get_orders	Connect Webull, see live positions and balances
3	Market Data	Polygon.io integration, real-time quotes, options chains + greeks, historical bars, local cache, Yahoo fallback	Fetch NVDA quote, options chain, 6-month history
4	Technicals	All indicators (MA, RSI, MACD, BB, ATR), trend detection, S/R calculation, volume analysis, pattern detection	Given NVDA data, output full technical profile
5	MCP + Sell	MCP server skeleton, wire Phase 1 tools, sell signal logic (trailing stop, S/R break, RSI, MA crossover)	Ask Claude: How's my portfolio? Should I sell anything?
6	LLM + Polish	Ollama setup (Qwen 14B), LLM service, NL summaries, error handling, logging, rate limiting, E2E testing	Full Phase 1 working via Claude Desktop

Phase 2: THINK (Weeks 7-14)

Week	Focus	Build	Test
7	Rules	strategy_weights seeded with 50 rules, market regime detection, IV rank calculation, rule matching	Given NVDA snapshot, system scores all strategies
8	Scanner	Universe loader, filter pipeline (cap>\$50M, price>\$5, OI threshold), scoring, ranking top 10-20	Run scanner, get ranked list of 15 tickers
9	Flow	Unusual Whales/FlowAlgo integration, unusual activity detection, flow direction tagging	Detect unusual call volume on AAPL, flag as bullish
10	Strategy	Strategy selection + contract picker, risk/reward calc, position sizing, local LLM reasoning	NVDA → Iron Condor: Sell 120P/Buy 115P/...
11	RAG	SEC EDGAR ingestion, earnings transcripts, chunking, local embedding, ChromaDB, query interface	Query 'NVDA outlook' → get relevant SEC chunks
12	Predict	Log every recommendation to predictions, full context capture, status management	5 recommendations, all properly logged
13	MCP Tools	Wire Phase 2 tools: discover_options, analyze_ticker, get_recommendation, explain_strategy, etc.	Ask Claude: What should I trade this week?
14	Monitor	Position Monitor (C13), active position tracking, alert triggers, Notification service (C14), position snapshots	Enter position, see alerts, close, outcome recorded

Phase 3: LEARN (Weeks 15-20)

Week	Focus	Build	Test
15-16	Learning	Outcome analysis pipeline, weight adjustment logic, learning_log tracking, min data thresholds	Simulate 50 trades, verify weights shift correctly
17-18	Profile	Execution behavior tracking, risk profile refinement, strategy fit analysis, auto-adjust recommendations	After 30+ trades, system identifies actual risk tolerance vs stated
19-20	Backtest	Backtesting engine (historical data), test rules against past 1-2 years, accuracy report, get_learnings tool	Backtest iron condor on high IV sideways for 2025

Phase 4: ACT + Dashboard (Weeks 21-24)

Week	Focus	Build	Test
21-22	Execute	Webull write scope re-auth, multi-leg order builder, approval workflow, execute_trade MCP tool	Recommend → approve → execute → fill confirmed
23	Dashboard	Next.js + MCP client, portfolio view, active positions, recommendations, alerts, WebSocket for real-time	Dashboard shows live data from same MCP server
24	Deploy	VPS deployment (Docker Compose), Traefik TLS, monitoring, scheduled jobs, multi-user testing, documentation	Full system 24/7 on VPS, multiple users, E2E working

Effort Estimate

Phase	Weeks	Hours/Week	Total Hours
Phase 1: SEE	6	10-15	60-90
Phase 2: THINK	8	10-15	80-120
Phase 3: LEARN	6	10-15	60-90
Phase 4: ACT + UI	4	10-15	40-60
Total	24		240-360 hours

At 10-15 hours per week (evenings + weekends), this is a 6-month project with something working and useful at the end of each phase.

What You Can Do After Each Phase

Phase	You Can Ask Claude
After Phase 1	'How's my portfolio? Should I sell anything?'
After Phase 2	'What should I trade this week?' → specific contracts
After Phase 3	'Which strategies are actually working for me?'

After Phase 4

'Execute the NVDA iron condor' → trade placed

16. Dashboard UI (Phase 4+)

Custom Next.js dashboard connects as MCP client to the same MCP server Claude uses. No REST API layer needed. WebSocket only for real-time alert push notifications.

Dashboard vs Claude: When to Use Which

Task	Claude Desktop	Dashboard
'What should I trade?'	Yes — conversational	No
'Explain this strategy'	Yes — needs reasoning	No
Check portfolio quickly	No — too slow	Yes — instant
Monitor active positions	No	Yes — real-time
View P&L chart	No	Yes — visual
Act on alert	No	Yes — one click
Weekly scan results	Either	Yes — table view
Deep analysis	Yes — complex reasoning	No
Enter/close position	Either	Yes — faster

User Tiers (Future Monetization)

Tier	Access	Cost
Free	Dashboard only (no Claude needed). Portfolio, recommendations, alerts, performance tracking.	Free
Pro	Dashboard + Claude Desktop. Conversational analysis, strategy explanations, complex ad-hoc questions.	Claude subscription

The MCP server is the product. Claude Desktop and Dashboard are just different windows into it.

17. Open Decisions

#	Decision	Options	Notes
1	MCP Server language	Python (FastMCP) vs Go	Python = faster dev, same ecosystem Go = better performance
2	VPS provider	DigitalOcean, Hetzner, Linode, AWS Lightsail	\$10-20/month. Need always-on for monitoring + scheduled scans
3	Options flow data	Unusual Whales (\$40/mo) vs FlowAlgo (\$70/mo)	UW has API. FlowAlgo may require scraping.
4	Alert delivery	Email + Slack + SMS	Start with email + Slack. Add SMS for critical only.
5	Backtesting engine	Build custom vs use backtrader/zipline	Custom needed for options-specific multi-leg backtesting
6	User onboarding	Self-service vs invite-only	Start invite-only (controlled), open registration later
7	Mobile app	React Native (future)	Phase 5+. Same MCP server, new client.

CONTINUATION INSTRUCTIONS FOR AI ASSISTANT:

This document is the COMPLETE BLUEPRINT v2.0. All design decisions are final and approved.

To start implementation: Begin with Phase 1, Week 1 — Postgres setup + Docker Compose + user auth + invite system. All 14 table schemas are defined in Section 6. Use Python as the primary language. Set up Docker Compose with Postgres, ChromaDB, and Ollama.

Key user context: Jaimin is a Staff Engineer at PayPal with deep Java/reactive programming experience. He prefers Python for this project. Primary broker is Webull. He wants the system to discover tickers autonomously (market cap > \$50M, no penny stocks). The self-learning loop (prediction tracking + outcome validation) is the most critical feature. Multi-user from day one. Claude Desktop is the only client for Phases 1-3. Dashboard added in Phase 4. Minimize cloud LLM costs — local Qwen 14B for analysis, cloud Claude only for conversation.